



Introduction to Algorithmic Techniques

Dr. Sumaiya Tabassum Nimi

Assistant Professor, Dept. of ECE — North South University

1. Why Algorithmic Techniques Matter

When solving computational problems, choosing the **right algorithmic strategy** often matters more than writing fast code. Different types of problems require different approaches, and understanding the major paradigms will help you design correct and efficient solutions.

This handout introduces five common techniques:

Greedy, Divide & Conquer, Dynamic Programming, Backtracking.

We also discuss *when* to apply each technique.

2. Greedy Algorithms

2.1 What is the Greedy Approach?

A greedy algorithm builds a solution step-by-step, choosing the **locally optimal** option at each stage with the hope of obtaining a global optimum.

Examples

- Activity Selection (choose earliest finishing)
- Fractional Knapsack (pick item with highest value/weight ratio)
- Dijkstra's Algorithm (select closest unvisited vertex)

Fun Example

You enter a bakery with only 100 Taka. You buy the most delicious-looking pastry **right now**, without planning ahead. You repeat this until your money runs out.

Greedy choice: *"This pastry looks the tastiest. I'm taking it!"*

Strength: Fast and simple. Weakness: You may end up with too many cupcakes and no chocolate cake.

When to Use Greedy?

Use Greedy algorithms when:

- Making the locally best choice always leads to the global optimum.
- The problem exhibits **optimal substructure** and **greedy-choice property**.
- A quick, efficient solution is needed and correctness can be proven.

3. Divide and Conquer

3.1 The Approach

Divide the problem into *smaller subproblems*, solve them recursively, and combine their results.

$$T(n) = aT(n/b) + f(n)$$

Examples

- Merge Sort, Quick Sort
- Binary Search
- Fast Exponentiation
- Strassen's Matrix Multiplication

Fun Example

You need to clean your messy room.

Strategy:

- Divide: Split the room into *the floor*, *table*, and *bed*.
- Conquer: Clean each separately.
- Combine: The whole room becomes clean.

Works great because cleaning small messes is easier than cleaning one huge mess.

When to Use Divide & Conquer?

Use Divide & Conquer when:

- The problem can be broken into independent smaller parts.
- Subproblems are of the same type as the original.
- There is a natural way to combine solutions.

4. Dynamic Programming (DP)

4.1 What is DP?

DP is used when the problem has:

- **Overlapping subproblems** (repeated computations)
- **Optimal substructure** (optimal solutions of subproblems form the global optimum)

DP avoids recomputation by storing results (memoization or tabulation).

Fun Example

You are watching a long TV series (100 episodes). Your friend asks you many times: “*What happened up to Episode 50?*”

Instead of rewatching 1–50 every time:

- You write short notes after each episode.
- When someone asks, you check your notes instead of rewatching.

DP idea: *Save work once → reuse it many times.*

Examples

- Fibonacci number computation
- 0/1 Knapsack
- Longest Common Subsequence (LCS)
- Matrix Chain Multiplication

When to Use DP?

Use DP when:

- The recursive solution repeats the same subproblems.
- Greedy does *not* work because a local choice may cause a future suboptimal choice.
- You can define a clear state and recurrence relation.

5. Backtracking

5.1 What is Backtracking?

A brute-force search technique that builds candidates and abandons (“backtracks”) whenever a partial solution cannot lead to a valid solution.

Examples

- N-Queens Problem
- Generating permutations
- Subset-sum search

Fun Example

You're choosing which outfit to wear. You try Shirt A → doesn't match the jeans. Try Shirt B → still bad. Try Shirt C → perfect!

Backtracking: try, regret, undo, try again.

When to Use Backtracking?

Use Backtracking when:

- The search space is large but can be pruned early.
- You need to explore all valid solutions (e.g., puzzles, combinatorial searches).

6. How to Choose the Right Technique?

Quick Guide

- **Greedy:** If local choices guarantee optimality.
- **Divide & Conquer:** If the problem naturally splits into smaller pieces.
- **Dynamic Programming:** If subproblems overlap and optimal substructure exists.
- **Backtracking:** If exploring a solution space with constraints.